

Solution Design Document

NutriTrack

Phase 2: Diabetes Risk Prediction

Machine Learning

Author: Abhijith Nallana

A self initiated personal project

Version 1.0 | July 2026

Contents

1. Version History
2. Executive Summary
3. Purpose of Solution Design Document
4. Project Background
5. Project Team
6. Point of Contact
7. Business Requirements
8. Scope
9. Success Criteria
10. Solution Overview
11. Automation Flow
12. Implementation Timeline
13. Technology Stack
14. Current State Process Flow
15. Future State Process Flow
16. Functional Requirements
17. Technical Requirements
18. Assumptions and Considerations
19. Risks and Dependencies
20. Appendix A. Model and Evaluation Results

Version History

Version	Date	Author	Description
1.0	July 2026	Abhijith Nallana	Initial draft covering Phase 2: feature engineering, the diabetes risk label, model training and comparison, and Streamlit integration.

Executive Summary

Phase 2 adds a supervised machine learning stage to NutriTrack. It reads the cleaned NHANES table produced in Phase 1 and predicts a person's diabetes risk category, Low, Medium, or High, from everyday information that needs no blood test: age, gender, body mass index, physical activity, and daily diet. The training label is defined from the American Diabetes Association fasting glucose cut points, and fasting glucose itself is used only to build the label, never as a model input, so the deployed tool functions as a no blood test screener.

This document describes the design of Phase 2, which delivers feature engineering, a three class risk label, a comparison of five classifiers with class weighting, an honest evaluation with imbalance aware metrics, and a saved model that a new Streamlit page uses to score a user's profile. Phase 2 is predictive rather than diagnostic. It does not diagnose diabetes and does not replace medical care.

Purpose of Solution Design Document

The purpose of this document is to define the scope, design, and technical approach for the Phase 2 machine learning work so that it is clear, repeatable, and straightforward to evaluate. It records the business and functional requirements, the label definition, the chosen models and metrics, the flow of data from the Phase 1 database to a saved model, the assumptions that shape the design, and the risks that need to be managed.

Project Background

Phase 1 produced a clean and unified store of USDA foods and NHANES respondents. That table pairs demographics, body measures, physical activity, and diet with a laboratory glucose measurement for the fasting subsample. This combination makes it possible to learn how lifestyle factors relate to a clinically grounded risk label. Type 2 diabetes often develops quietly and is undiagnosed until complications appear, so a simple screen that flags elevated risk from information a person already knows can prompt an earlier clinical check.

Project Team

NutriTrack is a self initiated personal project, designed and built from end to end by a single developer. There is no wider team. The project owner carries every role listed below.

- Data preparation and feature engineering: shaping the NHANES table into model features and the risk label.
- Modelling: training and comparing the classifiers and selecting the best by weighted F1.
- Integration: saving the model and scaler and wiring them into the Streamlit application.
- Documentation: preparing this design document.

Point of Contact

- Project owner and developer: Abhijith Nallana.

Business Requirements

The solution is expected to meet the following needs:

- Predict a diabetes risk category from features a person can supply without a blood test.
- Use a clinically grounded label so the target is meaningful rather than arbitrary.
- Report honest, imbalance aware metrics rather than raw accuracy alone.
- Keep the work reproducible, so that re running the notebook regenerates the same model.
- Integrate the model into the application as a risk assessment page.

Scope

In scope for Phase 2 is reading the Phase 1 NHANES table, engineering the ten model features, defining the three class risk label from the fasting glucose cut points, an eighty twenty stratified train and test split with standardisation fitted on the training set only, training and comparing five classifiers with class weighting, a light grid search on the random forest, evaluation with accuracy, precision, recall, and weighted and macro F1, feature importance, saving the best model and its scaler, and a Streamlit risk assessment page.

Out of scope for Phase 2 is any change to the Phase 1 data, deep learning, probability calibration, extensive hyperparameter search, the application of NHANES survey weights, and deployment to a public server. The generative AI assistant is delivered in Phase 3.

Success Criteria

Phase 2 is considered successful when the following are true:

- The pipeline runs from the Phase 1 database to a saved model and scaler without manual steps.
- The selected model clearly beats the naive baseline of always predicting the majority class.
- Every model is evaluated with imbalance aware metrics, and the results are interpreted.
- The risk assessment page loads the saved model and returns a category with a confidence value.
- The results are reproducible, so that re running the notebook regenerates the same model.

Solution Overview

Phase 2 keeps the clean separation between the data layer and the application layer. A Jupyter notebook reads the `nhanes` table from the Phase 1 SQLite database, engineers the features, builds the label, splits and scales the data, trains and compares the models, and writes the best model and its scaler to a `models` folder. The Streamlit application loads those two files and scores a user's profile on demand. Because the notebook is the only place that trains the model, the data work and the application work stay independent. The architecture is shown in Figure 1.

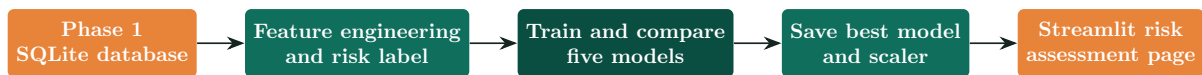


Figure 1. Phase 2 system architecture.

Automation Flow

The pipeline is automated from the database to the saved model. The notebook loads the survey table, creates the `gender_male` indicator and the feature matrix, labels each respondent from the fasting glucose cut points, splits and standardises the data, trains the five models, evaluates them, selects the best by weighted F1, and saves the model and scaler compressed to disk. The application loads those files the first time the risk page is opened and reuses them for every prediction. Re running the notebook refreshes the model. Figure 2 shows this flow.



Figure 2. Phase 2 automation flow.

Implementation Timeline

- **Phase 1, the data pipeline, exploratory analysis, and Streamlit application:** complete.
- **Phase 2, the diabetes risk model and its risk assessment page:** complete.
- **Phase 3, a generative AI nutrition assistant using retrieval augmented generation:** complete.

Technology Stack

- **Language and analysis:** Python, Pandas, and NumPy.
- **Machine learning:** scikit learn and XGBoost.
- **Model persistence:** joblib.
- **Visualization:** Plotly.
- **Storage:** SQLite, read from Phase 1.
- **Application:** Streamlit.
- **Notebook environment:** Jupyter.

Current State Process Flow

Before Phase 2 the application could describe nutrition and show diabetes patterns, but it could not estimate an individual's risk. A user who wanted a sense of their own risk had no path inside the tool. Figure 3 illustrates this current state.

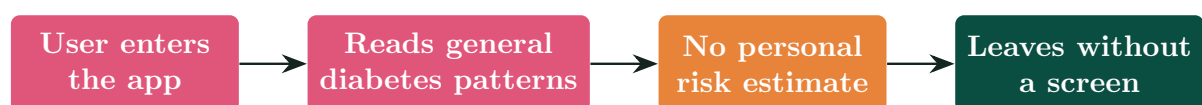


Figure 3. Current state process flow.

Future State Process Flow

In the future state the user enters their age, gender, body measures, activity, and daily diet on the risk assessment page and immediately receives a risk category, a confidence value, and a short list of contributing factors, all produced by the saved model. Figure 4 shows this target flow.



Figure 4. Future state process flow.

Functional Requirements

- **Risk label:** label each respondent Low, Medium, or High from the fasting glucose cut points below 100, 100 to 125, and 126 or above.
- **Feature set:** build ten non glucose features from the survey table.
- **Model training:** train logistic regression, a decision tree, a random forest, XGBoost, and a lightly tuned random forest, all with class weighting.
- **Evaluation:** report accuracy, precision, recall, and weighted and macro F1, and a confusion matrix for the best model.
- **Feature importance:** report which features drive the prediction.
- **Risk assessment page:** accept a user profile, scale it with the saved scaler, and return a category, a confidence value, and contributing factors.

Technical Requirements

- The notebook must read the Phase 1 database and must not modify it.
- The scaler must be fitted on the training set only and then applied to both sets.
- The best model and the scaler must be saved compressed to the `models` folder.
- The application must load the model in a read only manner and show a clear message if the model is missing.
- The solution must run on a standard Windows environment inside a Python virtual environment, and the environment must include XGBoost so the saved model loads.

Assumptions and Considerations

- Fasting glucose is used only to define the label, so the model never sees it as an input.
- The label is a proxy: it is the category a blood test would reveal, estimated from lifestyle alone, so moderate accuracy is expected and honest.
- Diet and diagnosis values in the survey are self reported, which introduces the normal effects of self reporting.
- The model is for personal and educational use only and does not provide medical diagnosis or treatment advice.

Risks and Dependencies

- **Class imbalance:** the three risk classes are uneven, with the High class smallest, which is why class weighting is used and why weighted and macro F1 matter more than accuracy.
- **Moderate accuracy:** predicting a lab defined label from lifestyle is a deliberately hard task, so the result is a useful screen rather than a precise test.
- **Library dependencies:** the solution relies on scikit learn, XGBoost, and joblib, which must be installed from the requirements file.
- **Data quality:** several survey fields contain missing values, which the pipeline fills with medians for the model features.

Appendix A. Model and Evaluation Results

The figures below are the actual results produced by the Phase 2 notebook. They are associations learned from the sample and should not be read as proof of cause.

Label distribution

The label is built on the fasting subsample, which gives 2,891 labeled respondents: 1,135 Low, 1,346 Medium, and 410 High. After an eighty twenty stratified split this is 2,312 training and 579 test rows.

Model comparison

All five models were trained with class weighting and evaluated on the held out test set. XGBoost was selected as the best model by weighted F1.

Model	Accuracy	Precision	Recall	F1 weighted	F1 macro
Logistic Regression	0.472	0.508	0.472	0.467	0.452
Decision Tree	0.423	0.479	0.423	0.420	0.407
Random Forest	0.484	0.513	0.484	0.488	0.462
XGBoost (selected)	0.513	0.525	0.513	0.516	0.493
Random Forest (tuned)	0.509	0.514	0.509	0.511	0.472

Interpretation

The selected model clearly beats the naive baseline of always predicting the majority Medium class, which sits near 0.47 accuracy. Because the label is defined by a laboratory value and predicted from lifestyle alone, accuracy is moderate rather than high, which is the honest and expected result for this kind of screening. Errors concentrate between neighbouring categories such as Low and Medium, which is sensible because those glucose ranges sit next to each other.

Feature importance

Age and body mass index are the strongest drivers of the predicted risk, followed by the dietary features. This matches the Phase 1 finding that age and BMI are the strongest non glucose correlates of diabetes in the sample.